as1

April 26, 2026

Road traffic collisions remain a significant public health and urban planning challenge in the United Kingdom. Between 2013 and 2022, over 1.6 million road casualty incidents were recorded, including 17,068 fatalities (ONS, 2023). In 2022 alone, 135,480 casualty accidents were reported, of which 29,742 resulted in death or serious injury, with 63% occurring on urban roads (ONS, 2023). Within this context, vulnerable road users (VRUs) — including pedestrians, cyclists, and motorcyclists — bear a disproportionate share of road risk. In 2022, pedestrians accounted for 22% of road deaths, motorcyclists 20%, and cyclists 5% (ONS, 2023). The rapid growth of platform-based gig economy services, particularly instant food delivery platforms such as Deliveroo and Uber Eats, has significantly increased the presence of two-wheelers on urban streets. Alongside the proliferation of shared e-scooters and e-bikes, these developments have intensified two-wheeler exposure in dense commercial areas, raising new concerns about rider safety in cities (Pirani et al., 2022). Despite growing investment in cycling infrastructure, cyclists and motorcyclists remain among the highest-risk road users in the UK. Between 2004 and 2021, cycling traffic in England grew by 62%, yet injury risk for cyclists remains substantially higher than for car occupants (DfT, 2022). Notably, cyclists have seen the smallest reduction in fatality rates among all road user groups in recent years (European Commission, 2024). Research consistently identifies temporal factors as significant predictors of collision severity for two-wheelers. Crashes occurring at night or in the early hours (22:00–06:00) are associated with greater injury severity for both cyclists and powered two-wheeler riders (Liu et al., 2022; Longo et al., 2024; Li et al., 2025). Spatial factors, particularly road type and junction characteristics, have also been shown to play an important role in determining collision outcomes. Studies of e-scooter and bicycle crashes in the UK found that approximately 66% of incidents occurred at or near junctions, with T-junctions and crossroads accounting for nearly half of all collisions (Useche et al., 2024). Road type has similarly been linked to severity, with dual carriageways and higher speed limit roads associated with increased injury severity among riders (Wahab & Jiang, 2019). Furthermore, land use context has emerged as a relevant factor: densely populated urban neighbourhoods and high-footfall commercial areas exhibit distinct spatial clustering of collisions compared to suburban or rural environments (Useche et al., 2024). Machine learning methods, particularly logistic regression and random forest classifiers, have been increasingly applied to predict road collision severity. A study of motorcycle crashes in Thailand found that random forest achieved an AUC of 0.768 in predicting fatal outcomes, outperforming seven other algorithms, while also highlighting the persistent challenge of class imbalance in severity prediction (Wahab & Jiang, 2019). Hybrid approaches combining tree-based models with interpretability tools such as SHAP values have also demonstrated strong performance in cyclist crash severity modelling (Wang et al., 2023). Building on this evidence, this study uses the UK Department for Transport's 2025 provisional road safety statistics, spatially joined with OpenStreetMap land use data, to examine how temporal patterns, road infrastructure characteristics, and fine-grained land use environments jointly influence the severity of two-wheeler collisions in London. By applying logistic regression, random forest classification, and SHAP analysis, this study aims to identify the

most important predictors of serious or fatal outcomes and to assess which factors offer the greatest potential for targeted safety interventions.

[1]:
```
# ============================================================
# Section 1: Import Libraries
# ============================================================
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.ticker as mtick
import seaborn as sns
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, roc_auc_score,␣
↪RocCurveDisplay
from sklearn.preprocessing import LabelEncoder
from imblearn.over_sampling import SMOTE
import shap
import warnings
warnings.filterwarnings('ignore')

print("Libraries loaded successfully")
```

```
Libraries loaded successfully
```

[2]:
```
!pip install imbalanced-learn shap --quiet
```

[3]:
```
# ============================================================
# Section 2: Load and Merge Data
# ============================================================
collision_url = "https://raw.githubusercontent.com/siyi-w/DSSS/main/as/
↪dft-road-casualty-statistics-collision-provisional-2025.csv"
vehicle_url   = "https://raw.githubusercontent.com/siyi-w/DSSS/main/as/
↪dft-road-casualty-statistics-vehicle-provisional-2025.csv"
casualty_url  = "https://raw.githubusercontent.com/siyi-w/DSSS/main/as/
↪dft-road-casualty-statistics-casualty-provisional-2025.csv"

df_collision = pd.read_csv(collision_url)
df_vehicle   = pd.read_csv(vehicle_url)
df_casualty  = pd.read_csv(casualty_url)

print(f"Collision records : {len(df_collision):,}")
print(f"Vehicle records   : {len(df_vehicle):,}")
print(f"Casualty records  : {len(df_casualty):,}")
```

```
Collision records : 48,472
Vehicle records   : 87,805
```

```
Casualty records  : 60,991
```

[4]:
```python
# ============================================================
# Section 3: Data Cleaning and Filtering
# ============================================================

# --- 3.1 Filter London (Metropolitan Police=1, City of London=2) ---
df_london = df_collision[df_collision['police_force'].isin([1, 2])].copy()
print(f"London collisions: {len(df_london):,}")

# --- 3.2 Filter two-wheelers ---
two_wheeler_types = [1, 2, 3, 4, 5, 23]
df_tw_vehicle = df_vehicle[df_vehicle['vehicle_type'].isin(two_wheeler_types)]
tw_ids = df_tw_vehicle['collision_index'].unique()

df_tw = df_london[df_london['collision_index'].isin(tw_ids)].copy()
print(f"London two-wheeler collisions: {len(df_tw):,}")

# --- 3.3 Binary severity target ---
df_tw['severity_binary'] = (df_tw['collision_severity'] <= 2).astype(int)
print(f"\nSeverity distribution:")
print(df_tw['severity_binary'].value_counts(normalize=True).round(3))

# --- 3.4 Time features ---
df_tw['date'] = pd.to_datetime(df_tw['date'], dayfirst=True)
df_tw['hour'] = df_tw['time'].str[:2].astype(float)
df_tw['is_weekend'] = (df_tw['day_of_week'] >= 5).astype(int)

# --- 3.5 Final modelling dataset ---
features = ['hour', 'is_weekend',
            'road_type', 'speed_limit', 'junction_detail',
            'light_conditions', 'weather_conditions',
            'road_surface_conditions', 'urban_or_rural_area',
            'number_of_vehicles', 'number_of_casualties']

df_model = df_tw[features + ['severity_binary']].dropna()
print(f"\nFinal modelling dataset: {len(df_model):,} rows")
print(f"Serious/Fatal rate: {df_model['severity_binary'].mean():.1%}")
```

```
London collisions: 10,062
London two-wheeler collisions: 4,966

Severity distribution:
severity_binary
0    0.796
1    0.204
Name: proportion, dtype: float64
```

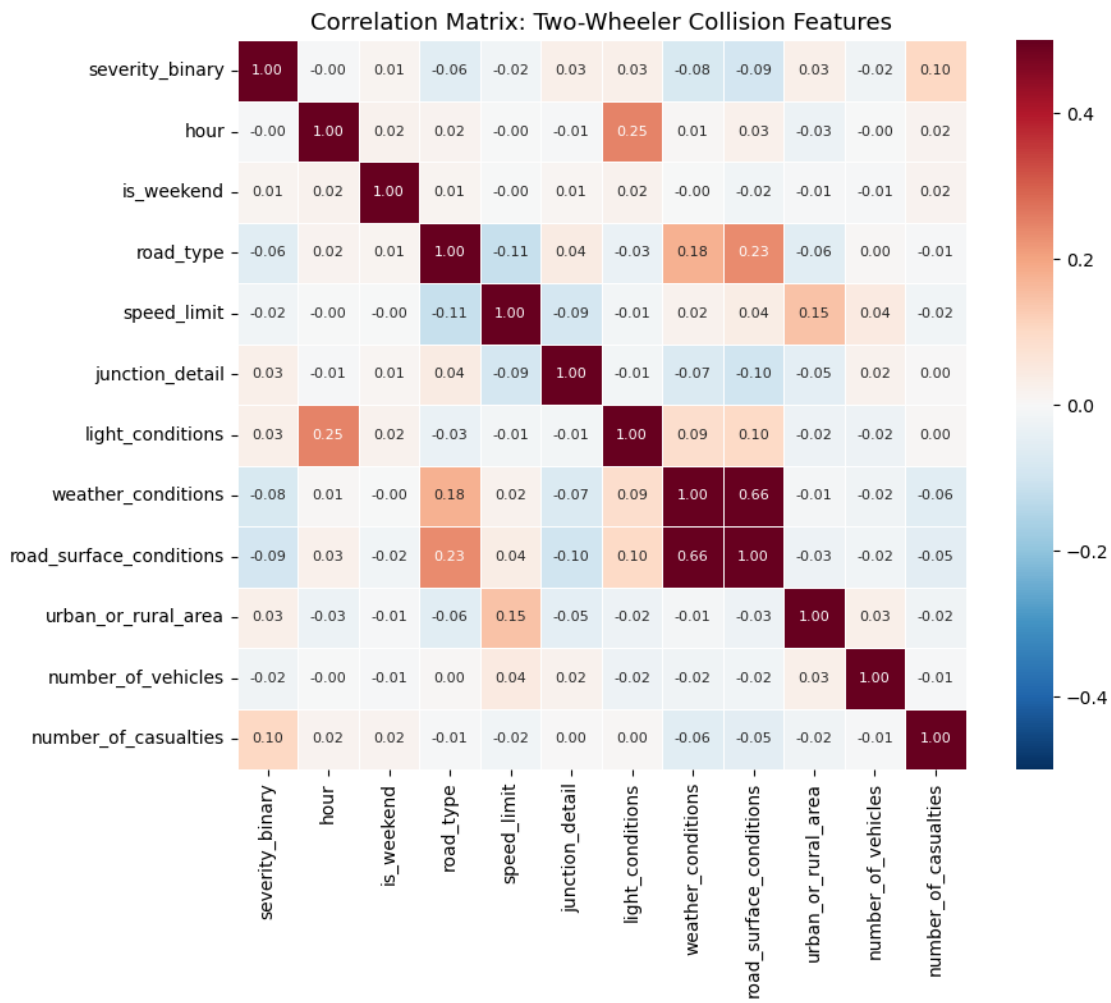
```
Final modelling dataset: 4,966 rows
Serious/Fatal rate: 20.4%
```

[5]:

```python
# ============================================================
# Section 4: EDA + Correlation Heatmap
# ============================================================

fig, axes = plt.subplots(2, 3, figsize=(16, 10))
fig.suptitle('Exploratory Analysis: Two-Wheeler Collision Severity in London␣
↪(2025)',
             fontsize=14, fontweight='bold', y=1.01)

# -- 4.1 Severity rate by hour --
hourly = df_model.groupby('hour')['severity_binary'].mean().reset_index()
axes[0,0].bar(hourly['hour'], hourly['severity_binary'], color='steelblue',␣
↪alpha=0.8)
axes[0,0].set_title('Serious/Fatal Rate by Hour of Day')
axes[0,0].set_xlabel('Hour'); axes[0,0].set_ylabel('Serious/Fatal Rate')
axes[0,0].yaxis.set_major_formatter(mtick.PercentFormatter(1.0))

# -- 4.2 Severity rate by day of week --
day_labels = ['Mon','Tue','Wed','Thu','Fri','Sat','Sun']
daily = df_tw.groupby('day_of_week')['severity_binary'].mean()
axes[0,1].bar(day_labels, daily.values, color='coral', alpha=0.8)
axes[0,1].set_title('Serious/Fatal Rate by Day of Week')
axes[0,1].set_xlabel('Day'); axes[0,1].set_ylabel('Serious/Fatal Rate')
axes[0,1].yaxis.set_major_formatter(mtick.PercentFormatter(1.0))

# -- 4.3 Severity rate by road type --
road_labels = {1:'Roundabout', 2:'One way', 3:'Dual carriageway',
               6:'Single carriageway', 7:'Slip road', 9:'Unknown'}
df_model['road_type_label'] = df_model['road_type'].map(road_labels).
↪fillna('Other')
road = df_model.groupby('road_type_label')['severity_binary'].mean().
↪sort_values(ascending=True)
axes[0,2].barh(road.index, road.values, color='teal', alpha=0.8)
axes[0,2].set_title('Serious/Fatal Rate by Road Type')
axes[0,2].set_xlabel('Serious/Fatal Rate')
axes[0,2].xaxis.set_major_formatter(mtick.PercentFormatter(1.0))

# -- 4.4 Severity rate by speed limit --
speed = df_model.groupby('speed_limit')['severity_binary'].mean().reset_index()
axes[1,0].bar(speed['speed_limit'].astype(str), speed['severity_binary'],
              color='mediumpurple', alpha=0.8)
axes[1,0].set_title('Serious/Fatal Rate by Speed Limit')
axes[1,0].set_xlabel('Speed Limit (mph)'); axes[1,0].set_ylabel('Serious/Fatal␣
↪Rate')
```

```
axes[1,0].yaxis.set_major_formatter(mtick.PercentFormatter(1.0))

# -- 4.5 Severity rate by light conditions --
light_labels = {1:'Daylight', 4:'Dark - lit', 5:'Dark - unlit', 6:'Dark - no␣
↪light', 7:'Dark - unknown'}
df_model['light_label'] = df_model['light_conditions'].map(light_labels).
↪fillna('Other')
light = df_model.groupby('light_label')['severity_binary'].mean().
↪sort_values(ascending=True)
axes[1,1].barh(light.index, light.values, color='darkorange', alpha=0.8)
axes[1,1].set_title('Serious/Fatal Rate by Light Conditions')
axes[1,1].set_xlabel('Serious/Fatal Rate')
axes[1,1].xaxis.set_major_formatter(mtick.PercentFormatter(1.0))

# -- 4.6 Collision count by hour (volume context) --
hour_count = df_model.groupby('hour').size()
axes[1,2].bar(hour_count.index, hour_count.values, color='slategray', alpha=0.8)
axes[1,2].set_title('Collision Count by Hour of Day')
axes[1,2].set_xlabel('Hour'); axes[1,2].set_ylabel('Number of Collisions')

plt.tight_layout()
plt.savefig('eda_severity.png', dpi=150, bbox_inches='tight')
plt.show()
```

[6]:
```
# -- 4.7 Correlation Heatmap --
fig, ax = plt.subplots(figsize=(10, 8))

corr_cols = ['severity_binary', 'hour', 'is_weekend',
             'road_type', 'speed_limit', 'junction_detail',
             'light_conditions', 'weather_conditions',
             'road_surface_conditions', 'urban_or_rural_area',
             'number_of_vehicles', 'number_of_casualties']

corr_matrix = df_model[corr_cols].corr()

sns.heatmap(corr_matrix,
            annot=True, fmt='.2f', cmap='RdBu_r', center=0,
            vmin=-0.5, vmax=0.5,
            square=True, linewidths=0.5, ax=ax,
            annot_kws={'size': 8})

ax.set_title('Correlation Matrix: Two-Wheeler Collision Features', fontsize=13)
plt.tight_layout()
plt.savefig('correlation_heatmap.png', dpi=150, bbox_inches='tight')
plt.show()

print("\nTop correlations with severity_binary:")
print(corr_matrix['severity_binary'].drop('severity_binary').abs()
      .sort_values(ascending=False).head(8).round(3))
```



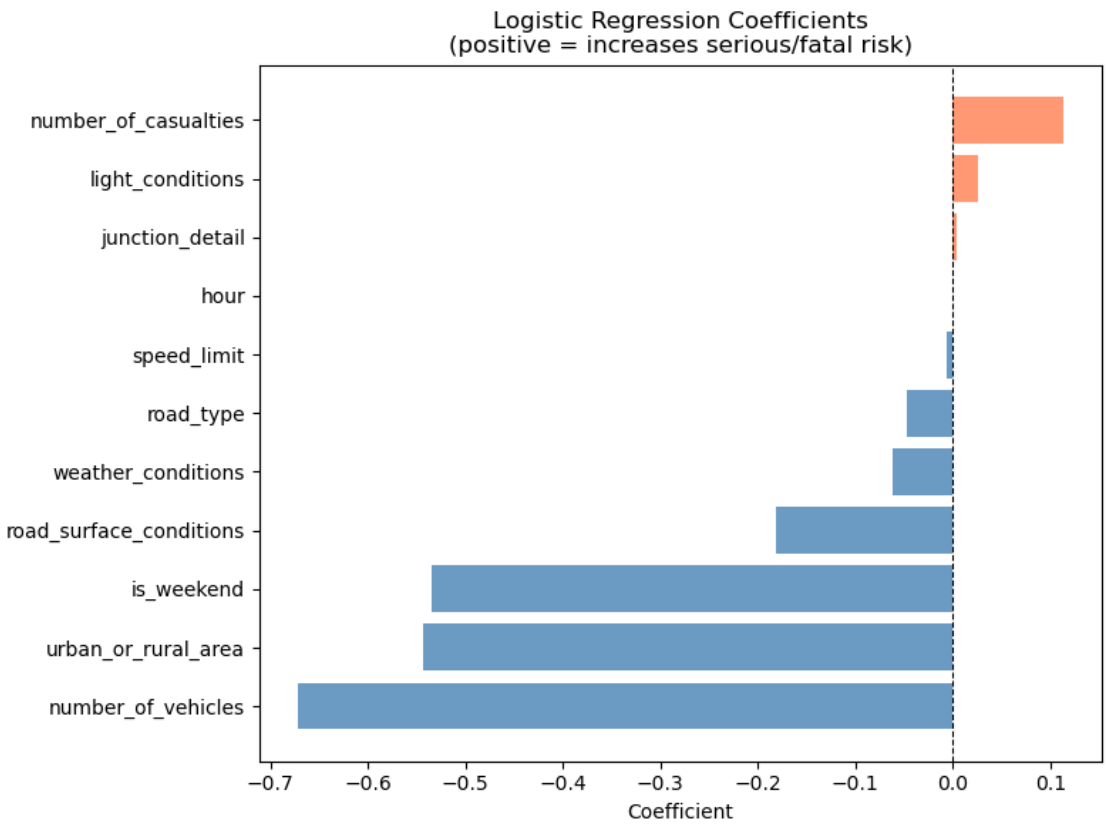

```
Top correlations with severity_binary:
number_of_casualties       0.104
road_surface_conditions    0.091
weather_conditions         0.080
road_type                  0.055
urban_or_rural_area        0.031
junction_detail            0.030
light_conditions           0.029
number_of_vehicles         0.021
Name: severity_binary, dtype: float64
```

[7]:
```
# ============================================================
# Section 5: Logistic Regression
# ============================================================
```

```python
X = df_model[features]
y = df_model['severity_binary']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

# SMOTE on training set only
smote = SMOTE(random_state=42)
X_train_sm, y_train_sm = smote.fit_resample(X_train, y_train)
print(f"After SMOTE - Training set class distribution: {pd.Series(y_train_sm).
 ↪value_counts().to_dict()}")

# Train logistic regression
lr = LogisticRegression(max_iter=1000, random_state=42)
lr.fit(X_train_sm, y_train_sm)

# Evaluate
y_pred_lr = lr.predict(X_test)
y_prob_lr = lr.predict_proba(X_test)[:, 1]

print("\n--- Logistic Regression ---")
print(classification_report(y_test, y_pred_lr, target_names=['Slight','Serious/
 ↪Fatal']))
print(f"ROC-AUC: {roc_auc_score(y_test, y_prob_lr):.3f}")

# Coefficient plot
coef_df = pd.DataFrame({
    'Feature': features,
    'Coefficient': lr.coef_[0]
}).sort_values('Coefficient')

fig, ax = plt.subplots(figsize=(8, 6))
colors = ['coral' if c > 0 else 'steelblue' for c in coef_df['Coefficient']]
ax.barh(coef_df['Feature'], coef_df['Coefficient'], color=colors, alpha=0.8)
ax.axvline(0, color='black', linewidth=0.8, linestyle='--')
ax.set_title('Logistic Regression Coefficients\n(positive = increases serious/
 ↪fatal risk)')
ax.set_xlabel('Coefficient')
plt.tight_layout()
plt.savefig('lr_coefficients.png', dpi=150, bbox_inches='tight')
plt.show()
```

```
After SMOTE - Training set class distribution: {0: 3163, 1: 3163}

--- Logistic Regression ---
               precision    recall  f1-score   support
```

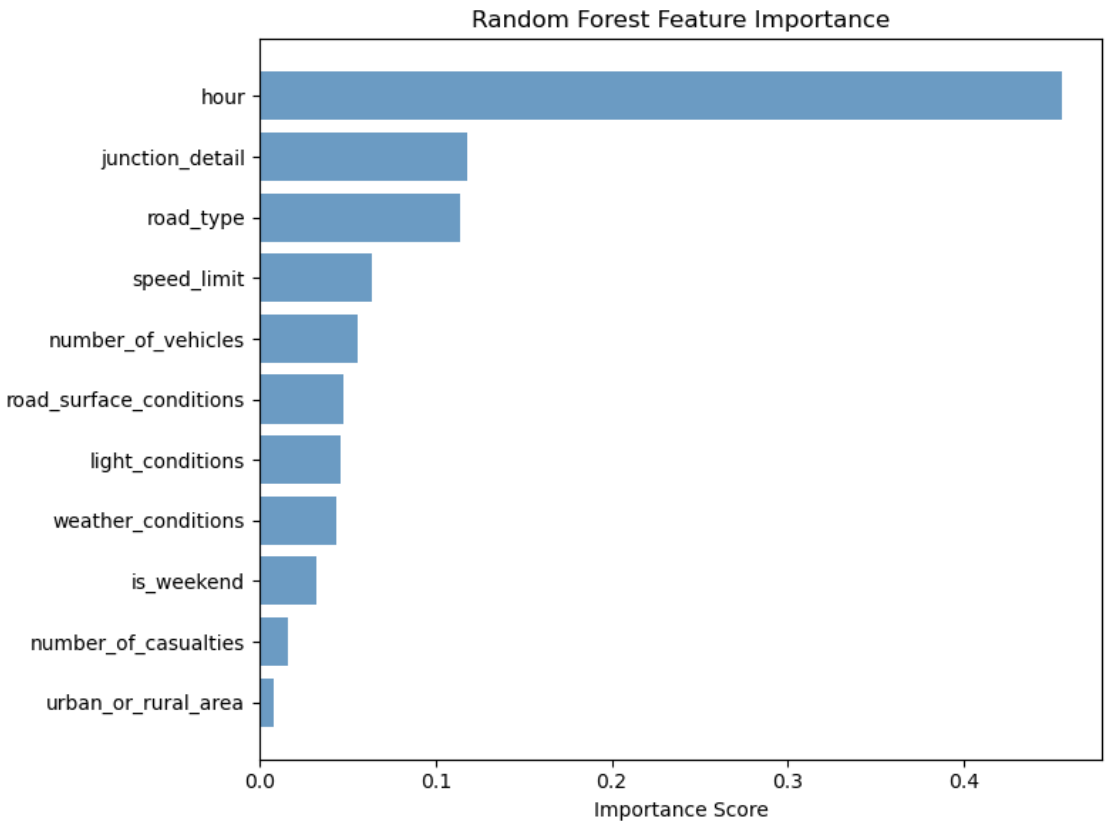
```
       Slight       0.82      0.55      0.66       791
Serious/Fatal       0.23      0.51      0.31       203

     accuracy                           0.54       994
    macro avg       0.52      0.53      0.49       994
 weighted avg       0.70      0.54      0.59       994

ROC-AUC: 0.557
```

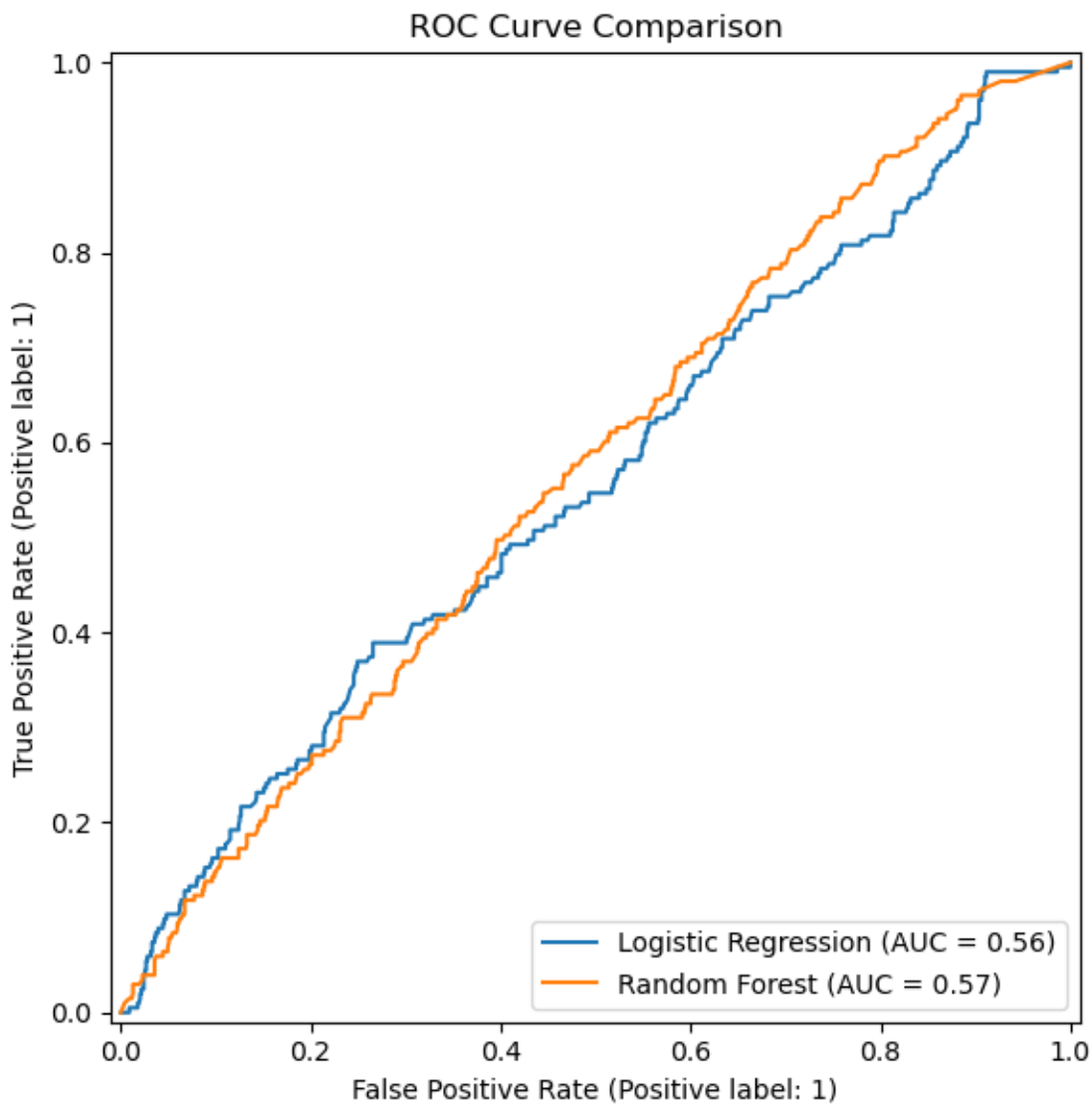


[8]:
```python
# ============================================================
# Section 6: Random Forest
# ============================================================

rf = RandomForestClassifier(n_estimators=100, random_state=42,␣
 ↪class_weight='balanced')
rf.fit(X_train_sm, y_train_sm)

y_pred_rf = rf.predict(X_test)
y_prob_rf = rf.predict_proba(X_test)[:, 1]

print("--- Random Forest ---")
```

```python
print(classification_report(y_test, y_pred_rf, target_names=['Slight','Serious/
↪Fatal']))
print(f"ROC-AUC: {roc_auc_score(y_test, y_prob_rf):.3f}")

# Feature importance plot
fi_df = pd.DataFrame({
    'Feature': features,
    'Importance': rf.feature_importances_
}).sort_values('Importance', ascending=True)

fig, ax = plt.subplots(figsize=(8, 6))
ax.barh(fi_df['Feature'], fi_df['Importance'], color='steelblue', alpha=0.8)
ax.set_title('Random Forest Feature Importance')
ax.set_xlabel('Importance Score')
plt.tight_layout()
plt.savefig('rf_importance.png', dpi=150, bbox_inches='tight')
plt.show()

# ROC curve comparison
fig, ax = plt.subplots(figsize=(7, 6))
RocCurveDisplay.from_predictions(y_test, y_prob_lr, name='Logistic Regression',␣
↪ax=ax)
RocCurveDisplay.from_predictions(y_test, y_prob_rf, name='Random Forest', ax=ax)
ax.set_title('ROC Curve Comparison')
plt.tight_layout()
plt.savefig('roc_comparison.png', dpi=150, bbox_inches='tight')
plt.show()
```

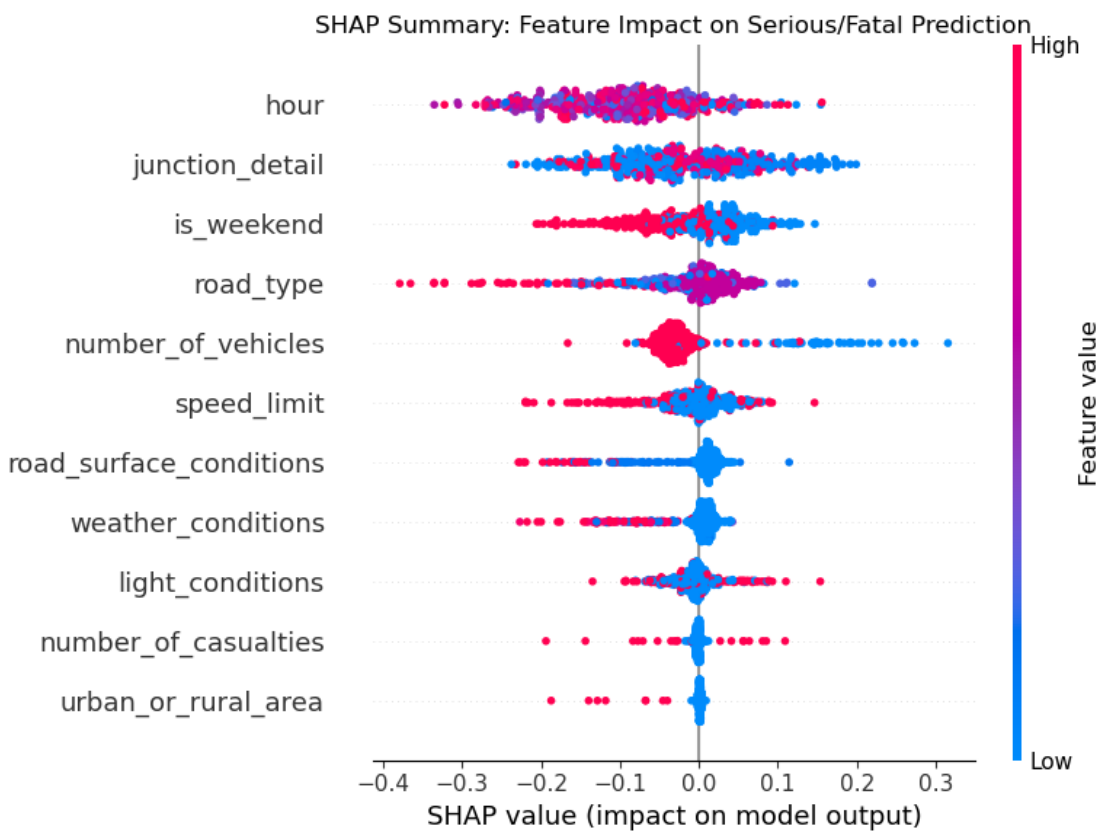
```
--- Random Forest ---
               precision    recall  f1-score   support

       Slight       0.81      0.77      0.79       791
Serious/Fatal       0.24      0.29      0.27       203

     accuracy                           0.67       994
    macro avg       0.53      0.53      0.53       994
 weighted avg       0.69      0.67      0.68       994

ROC-AUC: 0.570
```

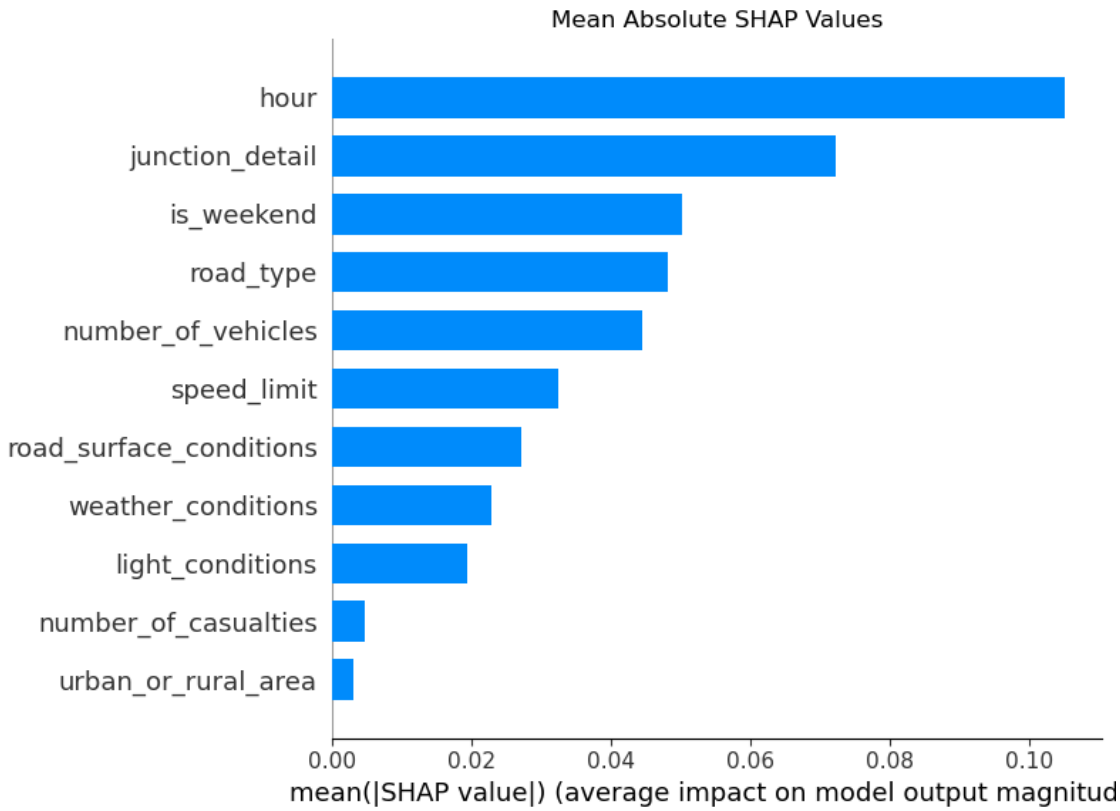

Random Forest Feature Importance
hour
junction_detail
road_type
speed_limit
number_of_vehicles
road_surface_conditions
light_conditions
weather_conditions
is_weekend
number_of_casualties
urban_or_rural_area
0.0
0.1
0.2
0.3
0.4
Importance Score



[9]:
```
# ============================================================
# Section 7: SHAP Analysis
# ============================================================
# Note: shap 0.51.0 - install via: pip install shap

explainer = shap.TreeExplainer(rf)
X_test_sample = X_test.sample(500, random_state=42)  # sample for speed
shap_values = explainer.shap_values(X_test_sample)

# SHAP values for class 1 (Serious/Fatal)
shap_vals_class1 = shap_values[:, :, 1] if shap_values.ndim == 3 else␣
↪shap_values[1]
```

```python
# -- 7.1 Beeswarm summary plot --
plt.figure()
shap.summary_plot(shap_vals_class1, X_test_sample,
                  feature_names=features, show=False)
plt.title('SHAP Summary: Feature Impact on Serious/Fatal Prediction')
plt.tight_layout()
plt.savefig('shap_summary.png', dpi=150, bbox_inches='tight')
plt.show()

# -- 7.2 Mean absolute SHAP bar chart --
plt.figure()
shap.summary_plot(shap_vals_class1, X_test_sample,
                  feature_names=features, plot_type='bar', show=False)
plt.title('Mean Absolute SHAP Values')
plt.tight_layout()
plt.savefig('shap_bar.png', dpi=150, bbox_inches='tight')
plt.show()
```





[10]:
```python
# ============================================================
# Section 3: Data Cleaning, Filtering and Spatial Join
# ============================================================
import geopandas as gpd
import zipfile, io, requests
import numpy as np

# --- 3.1 Filter London ---
df_london = df_collision[df_collision['police_force'].isin([1, 2])].copy()
print(f"London collisions: {len(df_london):,}")

# --- 3.2 Filter two-wheelers ---
two_wheeler_types = [1, 2, 3, 4, 5, 23]
df_tw_vehicle = df_vehicle[df_vehicle['vehicle_type'].isin(two_wheeler_types)]
tw_ids = df_tw_vehicle['collision_index'].unique()
df_tw = df_london[df_london['collision_index'].isin(tw_ids)].copy()
print(f"London two-wheeler collisions: {len(df_tw):,}")

# --- 3.3 Binary severity target ---
df_tw['severity_binary'] = (df_tw['collision_severity'] <= 2).astype(int)
```

```python
# --- 3.4 Time features ---
df_tw['date'] = pd.to_datetime(df_tw['date'], dayfirst=True)
df_tw['hour'] = df_tw['time'].str[:2].astype(float)
df_tw['is_weekend'] = (df_tw['day_of_week'] >= 5).astype(int)

# --- 3.5 OSM Spatial Join (Buffer    ) ---
zip_url = "https://raw.githubusercontent.com/siyi-w/DSSS/main/as/landuse/osm.
 ↪zip"
r = requests.get(zip_url)
z = zipfile.ZipFile(io.BytesIO(r.content))
z.extractall("/tmp/landuse/")

#
landuse = gpd.read_file("/tmp/landuse/gis_osm_landuse_a_free_1.shp",
                        columns=['fclass', 'geometry'])

#  landuse
landuse_map = {
    'retail': 'commercial', 'commercial': 'commercial',
    'industrial': 'industrial', 'residential': 'residential',
    'park': 'green', 'grass': 'green', 'forest': 'green',
    'meadow': 'green', 'scrub': 'green',
    'recreation_ground': 'green', 'farmland': 'green',
    'farmyard': 'green', 'heath': 'green',
    'allotments': 'green', 'cemetery': 'green',
}
landuse['landuse_class'] = landuse['fclass'].map(landuse_map).fillna('other')

#    landuse   other
landuse = landuse[landuse['landuse_class'] != 'other'].copy()
print(f"Landuse polygons (filtered): {len(landuse):,}")

#
gdf_tw = gpd.GeoDataFrame(
    df_tw.copy(),
    geometry=gpd.points_from_xy(df_tw['longitude'], df_tw['latitude']),
    crs='EPSG:4326'
).to_crs(epsg=27700)
landuse = landuse.to_crs(epsg=27700)

#  spatial join
joined = gpd.sjoin(gdf_tw, landuse[['geometry','landuse_class']],
                   how='left', predicate='within')
joined = joined[~joined.index.duplicated(keep='first')]

#    50
unmatched = joined[joined['landuse_class'].isna()].index
```

```python
print(f"  : {joined['landuse_class'].notna().sum():,},   : {len(unmatched):,}")

if len(unmatched) > 0:
    gdf_un = gdf_tw.loc[unmatched].copy()
    gdf_un['geometry'] = gdf_un.geometry.buffer(50)
    joined_buf = gpd.sjoin(gdf_un, landuse[['geometry','landuse_class']],
                           how='left', predicate='intersects')
    joined_buf = joined_buf[~joined_buf.index.duplicated(keep='first')]
    joined.loc[unmatched, 'landuse_class'] = joined_buf['landuse_class']

joined['landuse_class'] = joined['landuse_class'].fillna('other')
df_tw['landuse_class'] = joined['landuse_class'].values
print(f"\nLanduse :")
print(df_tw['landuse_class'].value_counts())

# --- 3.6 One-Hot Encoding ---
landuse_dummies = pd.get_dummies(df_tw['landuse_class'], prefix='lu').
 ↪astype(int)
df_tw = pd.concat([df_tw.reset_index(drop=True),
                   landuse_dummies.reset_index(drop=True)], axis=1)
landuse_cols = [c for c in df_tw.columns if c.startswith('lu_')]

# drop_first    lu_commercial
landuse_cols = [c for c in landuse_cols if c != 'lu_commercial']
print(f"Landuse dummy columns: {landuse_cols}")

# --- 3.7 Final modelling dataset ---
features = ['hour', 'is_weekend',
            'road_type', 'speed_limit', 'junction_detail',
            'light_conditions', 'weather_conditions',
            'road_surface_conditions', 'urban_or_rural_area',
            'number_of_vehicles', 'number_of_casualties'] + landuse_cols

df_model = df_tw[features + ['severity_binary']].dropna()
print(f"\nFinal modelling dataset: {len(df_model):,} rows")
print(f"Serious/Fatal rate: {df_model['severity_binary'].mean():.1%}")
```

```
London collisions: 10,062
London two-wheeler collisions: 4,966
Landuse polygons (filtered): 79,672
  : 1,035,   : 3,931

Landuse :
landuse_class
residential    2984
commercial     1046
green           694
industrial      124
```

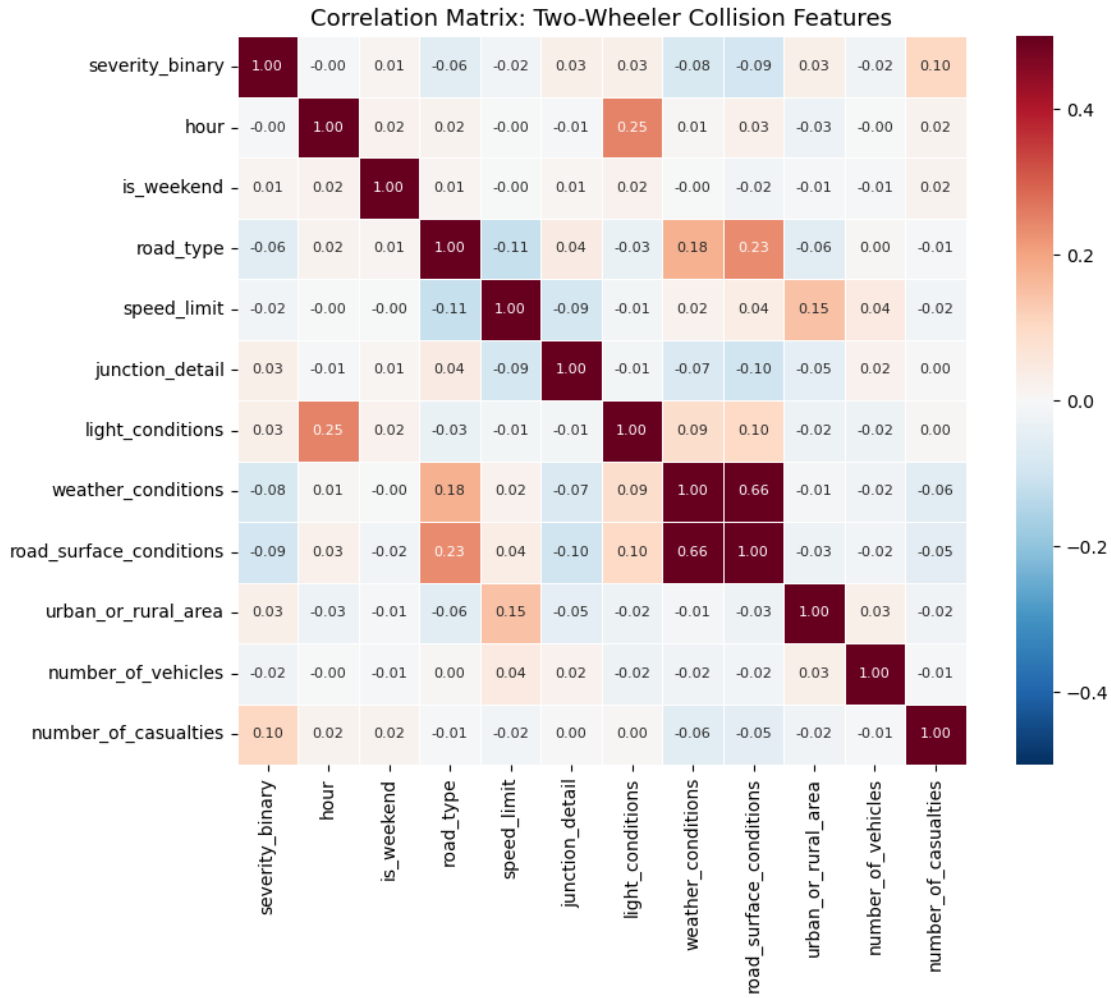
```
other           118
Name: count, dtype: int64
Landuse dummy columns: ['lu_green', 'lu_industrial', 'lu_other',
'lu_residential']

Final modelling dataset: 4,966 rows
Serious/Fatal rate: 20.4%
```

[11]:
```
# ============================================================
# Section 4: EDA + Correlation Heatmap
# ============================================================

fig, axes = plt.subplots(2, 3, figsize=(16, 10))
fig.suptitle('Exploratory Analysis: Two-Wheeler Collision Severity in London␣
↪(2025)',
             fontsize=14, fontweight='bold', y=1.01)

# -- 4.1 Severity rate by hour --
hourly = df_model.groupby('hour')['severity_binary'].mean().reset_index()
axes[0,0].bar(hourly['hour'], hourly['severity_binary'], color='steelblue',␣
↪alpha=0.8)
axes[0,0].set_title('Serious/Fatal Rate by Hour of Day')
axes[0,0].set_xlabel('Hour'); axes[0,0].set_ylabel('Serious/Fatal Rate')
axes[0,0].yaxis.set_major_formatter(mtick.PercentFormatter(1.0))

# -- 4.2 Severity rate by day of week --
day_labels = ['Mon','Tue','Wed','Thu','Fri','Sat','Sun']
daily = df_tw.groupby('day_of_week')['severity_binary'].mean()
axes[0,1].bar(day_labels, daily.values, color='coral', alpha=0.8)
axes[0,1].set_title('Serious/Fatal Rate by Day of Week')
axes[0,1].set_xlabel('Day'); axes[0,1].set_ylabel('Serious/Fatal Rate')
axes[0,1].yaxis.set_major_formatter(mtick.PercentFormatter(1.0))

# -- 4.3 Severity rate by road type --
road_labels = {1:'Roundabout', 2:'One way', 3:'Dual carriageway',
               6:'Single carriageway', 7:'Slip road', 9:'Unknown'}
df_model['road_type_label'] = df_model['road_type'].map(road_labels).
↪fillna('Other')
road = df_model.groupby('road_type_label')['severity_binary'].mean().
↪sort_values(ascending=True)
axes[0,2].barh(road.index, road.values, color='teal', alpha=0.8)
axes[0,2].set_title('Serious/Fatal Rate by Road Type')
axes[0,2].set_xlabel('Serious/Fatal Rate')
axes[0,2].xaxis.set_major_formatter(mtick.PercentFormatter(1.0))

# -- 4.4 Severity rate by speed limit --
speed = df_model.groupby('speed_limit')['severity_binary'].mean().reset_index()
```

```
axes[1,0].bar(speed['speed_limit'].astype(str), speed['severity_binary'],
              color='mediumpurple', alpha=0.8)
axes[1,0].set_title('Serious/Fatal Rate by Speed Limit')
axes[1,0].set_xlabel('Speed Limit (mph)'); axes[1,0].set_ylabel('Serious/Fatal␣
 ↪Rate')
axes[1,0].yaxis.set_major_formatter(mtick.PercentFormatter(1.0))

# -- 4.5 Severity rate by light conditions --
light_labels = {1:'Daylight', 4:'Dark - lit', 5:'Dark - unlit', 6:'Dark - no␣
 ↪light', 7:'Dark - unknown'}
df_model['light_label'] = df_model['light_conditions'].map(light_labels).
 ↪fillna('Other')
light = df_model.groupby('light_label')['severity_binary'].mean().
 ↪sort_values(ascending=True)
axes[1,1].barh(light.index, light.values, color='darkorange', alpha=0.8)
axes[1,1].set_title('Serious/Fatal Rate by Light Conditions')
axes[1,1].set_xlabel('Serious/Fatal Rate')
axes[1,1].xaxis.set_major_formatter(mtick.PercentFormatter(1.0))

# -- 4.6 Collision count by hour (volume context) --
hour_count = df_model.groupby('hour').size()
axes[1,2].bar(hour_count.index, hour_count.values, color='slategray', alpha=0.8)
axes[1,2].set_title('Collision Count by Hour of Day')
axes[1,2].set_xlabel('Hour'); axes[1,2].set_ylabel('Number of Collisions')

plt.tight_layout()
plt.savefig('eda_severity.png', dpi=150, bbox_inches='tight')
plt.show()
```

[12]:
```
# -- 4.7 Correlation Heatmap --
fig, ax = plt.subplots(figsize=(10, 8))

corr_cols = ['severity_binary', 'hour', 'is_weekend',
             'road_type', 'speed_limit', 'junction_detail',
             'light_conditions', 'weather_conditions',
             'road_surface_conditions', 'urban_or_rural_area',
             'number_of_vehicles', 'number_of_casualties']

corr_matrix = df_model[corr_cols].corr()

sns.heatmap(corr_matrix,
            annot=True, fmt='.2f', cmap='RdBu_r', center=0,
            vmin=-0.5, vmax=0.5,
            square=True, linewidths=0.5, ax=ax,
            annot_kws={'size': 8})

ax.set_title('Correlation Matrix: Two-Wheeler Collision Features', fontsize=13)
plt.tight_layout()
plt.savefig('correlation_heatmap.png', dpi=150, bbox_inches='tight')
plt.show()

print("\nTop correlations with severity_binary:")
print(corr_matrix['severity_binary'].drop('severity_binary').abs()
```

```
    .sort_values(ascending=False).head(8).round(3))
```



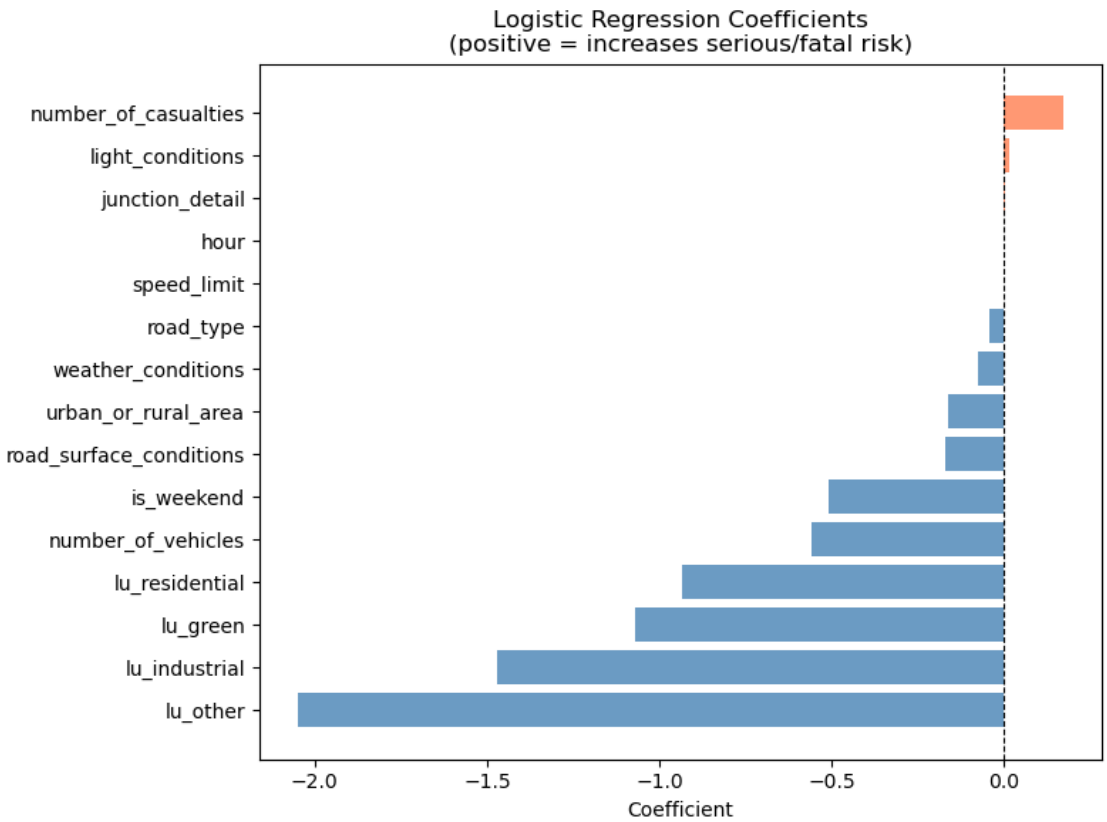

```
Top correlations with severity_binary:
number_of_casualties        0.104
road_surface_conditions     0.091
weather_conditions          0.080
road_type                   0.055
urban_or_rural_area         0.031
junction_detail             0.030
light_conditions            0.029
number_of_vehicles          0.021
Name: severity_binary, dtype: float64
```

[13]:
```
# ============================================================
# Section 5: Logistic Regression
```

```python
# ==============================================================

X = df_model[features].copy()

#
# One-Hot bool  int
for col in landuse_cols:
    X[col] = X[col].astype(int)

y = df_model['severity_binary']

print("Feature dtypes:")
print(X.dtypes)
print(f"\nX shape: {X.shape}")

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

# SMOTE on training set only
smote = SMOTE(random_state=42)
X_train_sm, y_train_sm = smote.fit_resample(X_train, y_train)
print(f"After SMOTE - Training set class distribution: {pd.Series(y_train_sm).
 ↪value_counts().to_dict()}")

# Train logistic regression
lr = LogisticRegression(max_iter=1000, random_state=42)
lr.fit(X_train_sm, y_train_sm)

# Evaluate
y_pred_lr = lr.predict(X_test)
y_prob_lr = lr.predict_proba(X_test)[:, 1]

print("\n--- Logistic Regression ---")
print(classification_report(y_test, y_pred_lr, target_names=['Slight','Serious/
 ↪Fatal']))
print(f"ROC-AUC: {roc_auc_score(y_test, y_prob_lr):.3f}")

# Coefficient plot
coef_df = pd.DataFrame({
    'Feature': features,
    'Coefficient': lr.coef_[0]
}).sort_values('Coefficient')

fig, ax = plt.subplots(figsize=(8, 6))
colors = ['coral' if c > 0 else 'steelblue' for c in coef_df['Coefficient']]
ax.barh(coef_df['Feature'], coef_df['Coefficient'], color=colors, alpha=0.8)
ax.axvline(0, color='black', linewidth=0.8, linestyle='--')
```

```
ax.set_title('Logistic Regression Coefficients\n(positive = increases serious/
 ↪fatal risk)')
ax.set_xlabel('Coefficient')
plt.tight_layout()
plt.savefig('lr_coefficients.png', dpi=150, bbox_inches='tight')
plt.show()
```

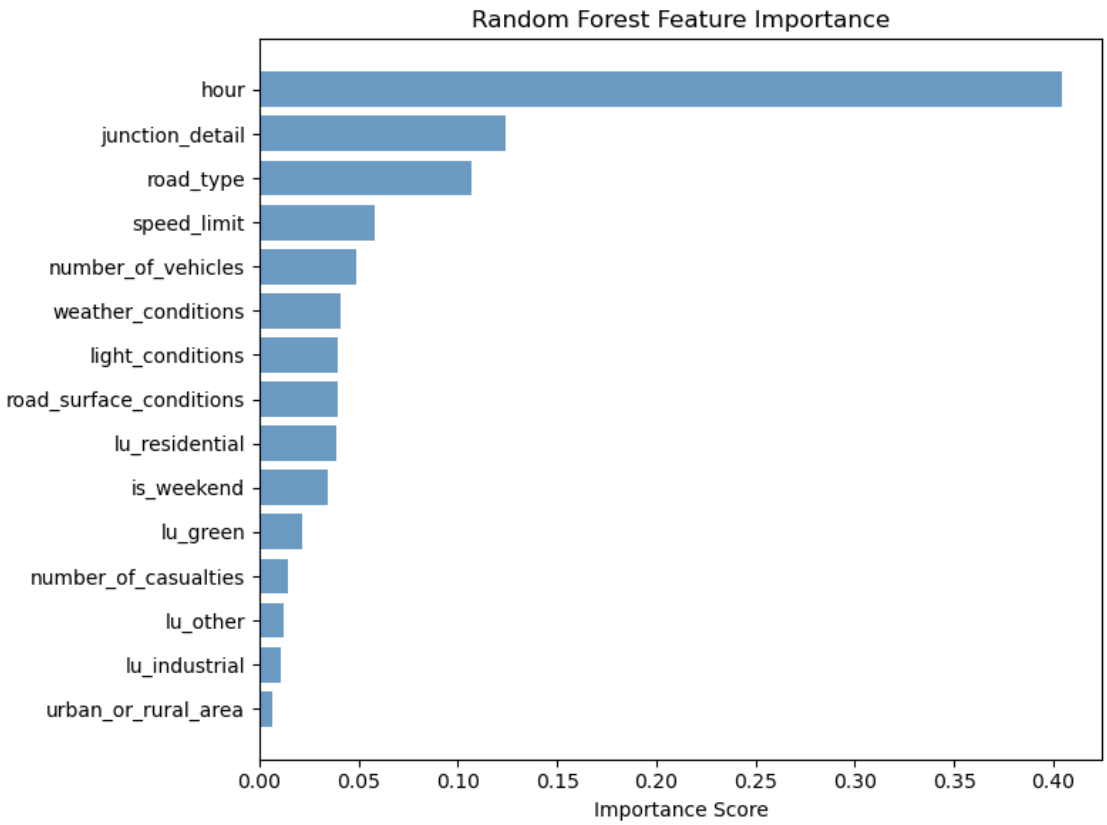
```
Feature dtypes:
hour                        float64
is_weekend                    int64
road_type                     int64
speed_limit                   int64
junction_detail               int64
light_conditions              int64
weather_conditions            int64
road_surface_conditions       int64
urban_or_rural_area           int64
number_of_vehicles            int64
number_of_casualties          int64
lu_green                      int64
lu_industrial                 int64
lu_other                      int64
lu_residential                int64
dtype: object

X shape: (4966, 15)
After SMOTE - Training set class distribution: {0: 3163, 1: 3163}

--- Logistic Regression ---
               precision    recall  f1-score   support

       Slight       0.81      0.72      0.76       791
Serious/Fatal       0.23      0.33      0.27       203

     accuracy                           0.64       994
    macro avg       0.52      0.52      0.51       994
 weighted avg       0.69      0.64      0.66       994

ROC-AUC: 0.539
```

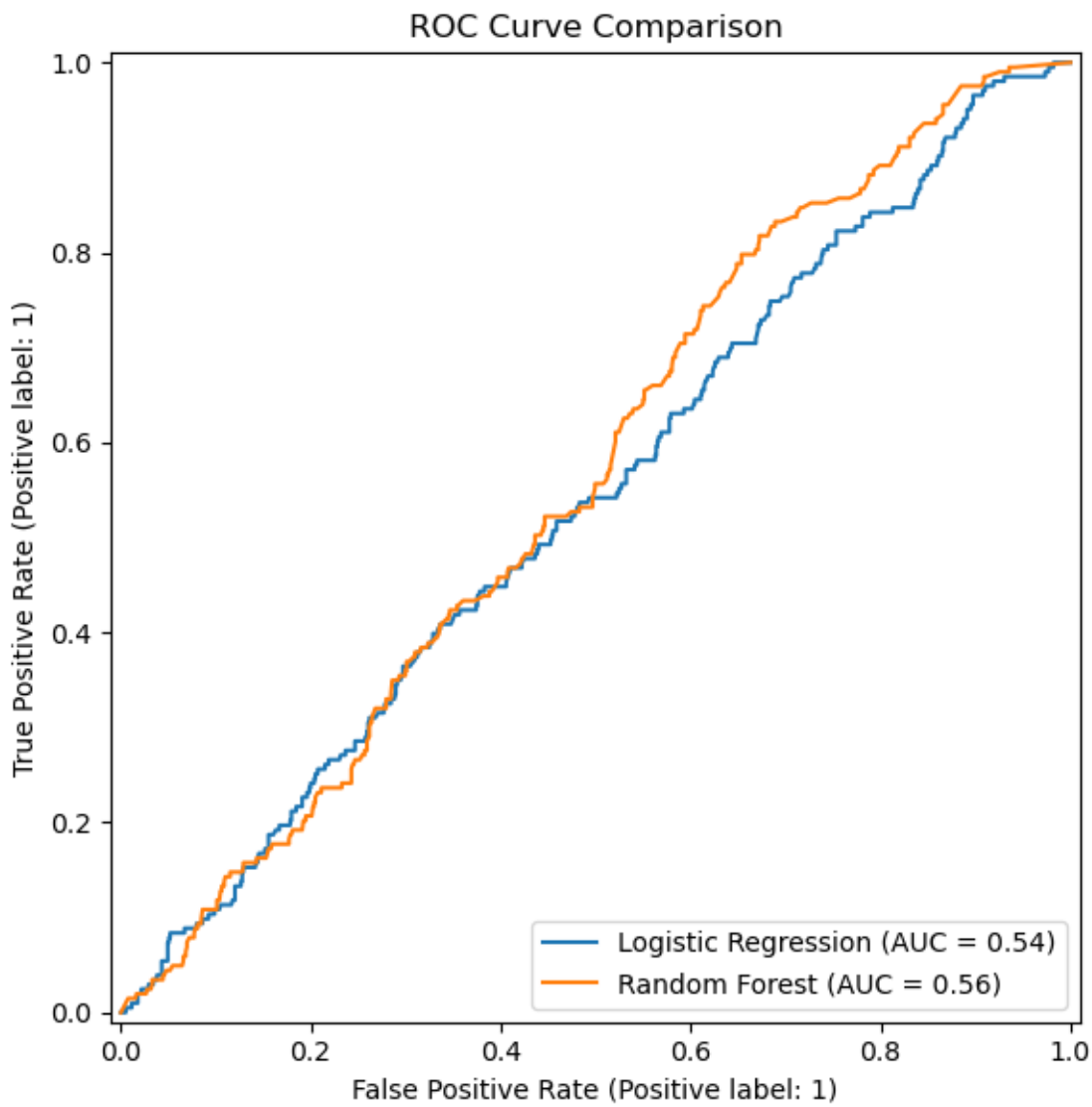


[14]:
```
# ============================================================
# Section 6: Random Forest
# ============================================================

rf = RandomForestClassifier(n_estimators=100, random_state=42,␣
↪class_weight='balanced')
rf.fit(X_train_sm, y_train_sm)

y_pred_rf = rf.predict(X_test)
y_prob_rf = rf.predict_proba(X_test)[:, 1]

print("--- Random Forest ---")
print(classification_report(y_test, y_pred_rf, target_names=['Slight','Serious/
↪Fatal']))
print(f"ROC-AUC: {roc_auc_score(y_test, y_prob_rf):.3f}")

# Feature importance plot
fi_df = pd.DataFrame({
    'Feature': features,
    'Importance': rf.feature_importances_
```

```python
}).sort_values('Importance', ascending=True)

fig, ax = plt.subplots(figsize=(8, 6))
ax.barh(fi_df['Feature'], fi_df['Importance'], color='steelblue', alpha=0.8)
ax.set_title('Random Forest Feature Importance')
ax.set_xlabel('Importance Score')
plt.tight_layout()
plt.savefig('rf_importance.png', dpi=150, bbox_inches='tight')
plt.show()

# ROC curve comparison
fig, ax = plt.subplots(figsize=(7, 6))
RocCurveDisplay.from_predictions(y_test, y_prob_lr, name='Logistic Regression',␣
↪ax=ax)
RocCurveDisplay.from_predictions(y_test, y_prob_rf, name='Random Forest', ax=ax)
ax.set_title('ROC Curve Comparison')
plt.tight_layout()
plt.savefig('roc_comparison.png', dpi=150, bbox_inches='tight')
plt.show()
```

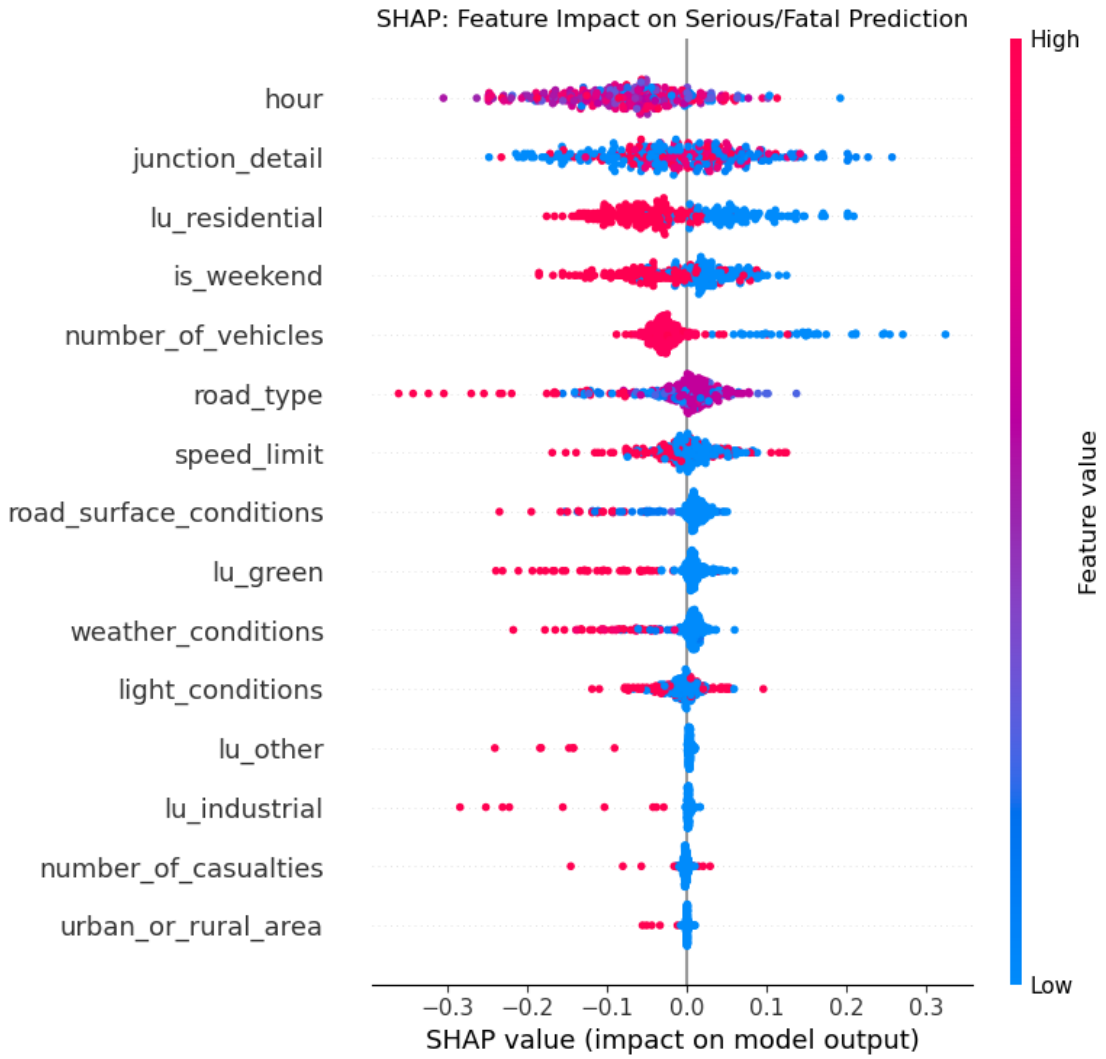
```
--- Random Forest ---
               precision    recall  f1-score   support

       Slight       0.80      0.75      0.78       791
Serious/Fatal       0.21      0.26      0.24       203

     accuracy                           0.65       994
    macro avg       0.51      0.51      0.51       994
 weighted avg       0.68      0.65      0.67       994

ROC-AUC: 0.562
```

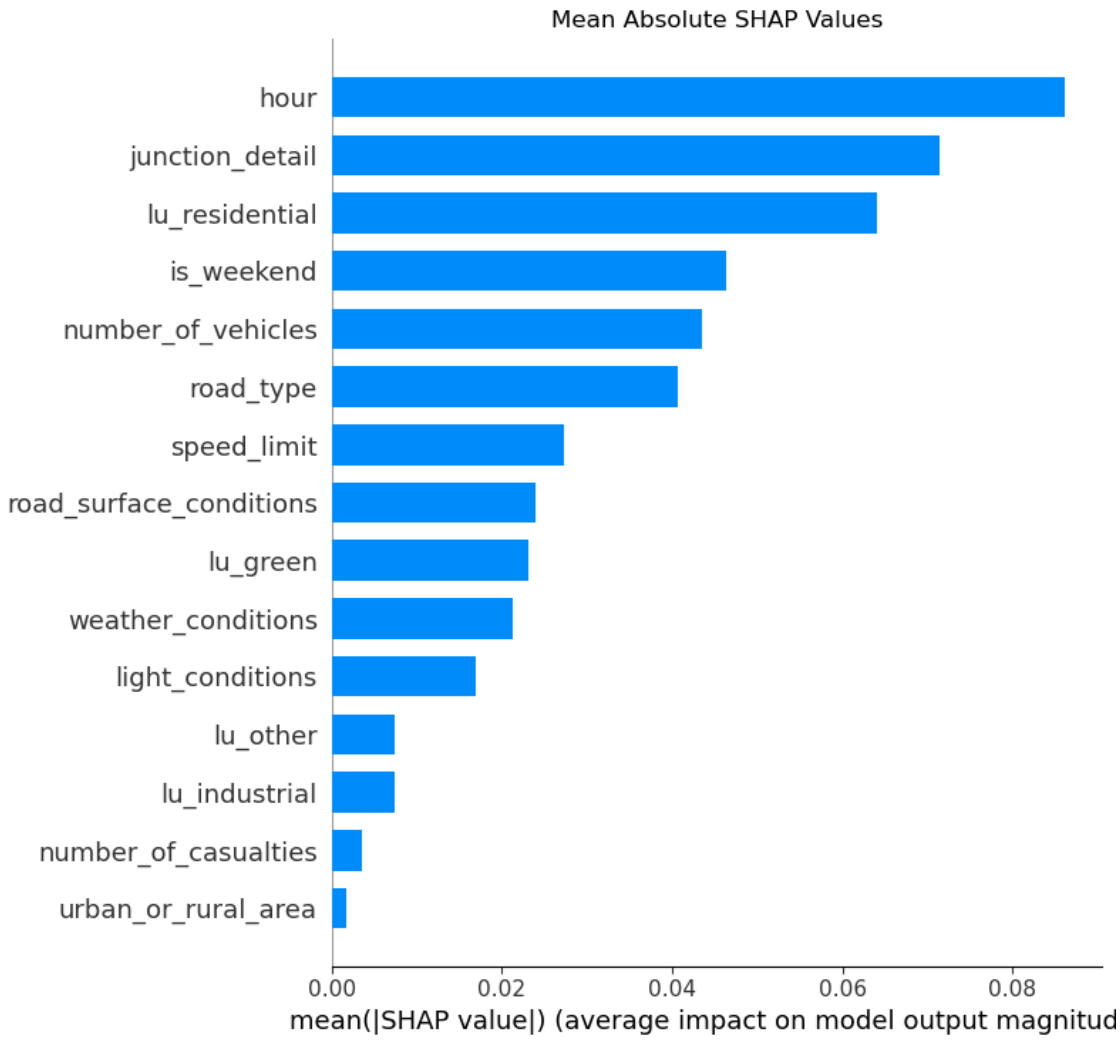

Random Forest Feature Importance
hour
junction_detail
road_type
speed_limit
number_of_vehicles
weather_conditions
light_conditions
road_surface_conditions
lu_residential
is_weekend
lu_green
number_of_casualties
lu_other
lu_industrial
urban_or_rural_area
0.00
0.05
0.10
0.15
0.20
0.25
0.30
0.35
0.40
Importance Score



[15]:
```
# ============================================================
# Section 7: SHAP Analysis
# ============================================================
explainer = shap.TreeExplainer(rf)
X_test_sample = X_test.sample(300, random_state=42)
shap_values = explainer.shap_values(X_test_sample)

#  shap 0.51.0
if isinstance(shap_values, list):
    shap_vals = shap_values[1]  # class 1 = Serious/Fatal
else:
    shap_vals = shap_values[:, :, 1]
```

```python
# -- 7.1 Beeswarm plot --
plt.figure()
shap.summary_plot(shap_vals, X_test_sample,
                  feature_names=features, show=False)
plt.title('SHAP: Feature Impact on Serious/Fatal Prediction')
plt.tight_layout()
plt.savefig('shap_summary.png', dpi=150, bbox_inches='tight')
plt.show()

# -- 7.2 Bar plot --
plt.figure()
shap.summary_plot(shap_vals, X_test_sample,
                  feature_names=features, plot_type='bar', show=False)
plt.title('Mean Absolute SHAP Values')
plt.tight_layout()
plt.savefig('shap_bar.png', dpi=150, bbox_inches='tight')
plt.show()
```


SHAP: Feature Impact on Serious/Fatal Prediction
hour
junction_detail
lu_residential
is_weekend
number_of_vehicles
road_type
speed_limit
road_surface_conditions
lu_green
weather_conditions
light_conditions
lu_other
lu_industrial
number_of_casualties
urban_or_rural_area
−0.3
−0.2
−0.1
0.0
0.1
0.2
0.3
SHAP value (impact on model output)
High
Feature value
Low


Mean Absolute SHAP Values
hour
junction_detail
lu_residential
is_weekend
number_of_vehicles
road_type
speed_limit
road_surface_conditions
lu_green
weather_conditions
light_conditions
lu_other
lu_industrial
number_of_casualties
urban_or_rural_area
0.00
0.02
0.04
0.06
0.08
mean(|SHAP value|) (average impact on model output magnitud

[ ]:

[ ]: